

5

Resampling Methods

Resampling methods là một tool không thể thiếu trong thống kê hiện đại. Các method này liên quan đến việc lặp đi lặp lại quá trình lấy sample từ một training set và fit lại model quan tâm trên từng sample để thu được thêm thông tin về fitted model. Chẳng hạn, để ước lượng variability của một linear regression fit, ta có thể liên tục lấy các sample khác nhau từ training data, fit linear regression trên từng sample mới, rồi xem mức độ khác nhau giữa các fit thu được. Cách làm như vậy có thể cung cấp những thông tin mà ta không thể có được nếu chỉ fit model một lần trên training sample ban đầu.

Các resampling approach có thể tốn nhiều computation, bởi chúng đòi hỏi cùng một statistical method phải được fit nhiều lần trên các subset khác nhau của training data. Tuy nhiên, nhờ những tiến bộ gần đây về computing power, yêu cầu tính toán của resampling methods nhìn chung không còn quá nặng nề. Trong chương này, ta thảo luận hai resampling method được dùng phổ biến nhất: *cross-validation* và *bootstrap*. Cả hai đều là những tool quan trọng trong ứng dụng thực tế của nhiều statistical learning procedure. Ví dụ, cross-validation có thể được dùng để ước lượng test error gắn với một statistical learning method nhằm đánh giá performance của method đó, hoặc để chọn mức flexibility thích hợp. Quá trình đánh giá performance của một model được gọi là *model assessment*, còn quá trình chọn mức flexibility phù hợp cho model được gọi là *model selection*. Bootstrap được dùng trong nhiều bối cảnh, phổ biến nhất là để cung cấp một thước đo về accuracy của một parameter estimate hoặc của một statistical learning method cho trước.

model assessment
model selection

5.1 Cross-Validation

Trong Chapter 2, ta đã thảo luận sự khác biệt giữa test error rate và training error rate. Test error là average error phát sinh khi dùng một statistical learning method để dự đoán response trên một observation mới — tức một measurement không được dùng khi training method. Với một dataset cho trước, việc sử dụng một statistical learning method chỉ thực sự hợp lý nếu method đó tạo ra test error thấp. Test error có thể tính dễ dàng nếu ta có một test set được dành riêng. Đáng tiếc, trường hợp này thường không xảy ra. Ngược lại, training error có thể tính dễ dàng bằng cách áp dụng statistical learning method lên chính các observation đã được dùng để training. Nhưng như ta đã thấy ở Chapter 2, training error rate thường khác đáng kể so với test error rate, và đặc biệt là nó có thể đánh giá thấp test error một cách nghiêm trọng.

Khi không có một test set riêng đủ lớn để ước lượng trực tiếp test error rate, ta có thể dùng nhiều technique khác nhau để ước lượng đại lượng này từ training data hiện có. Một số method thực hiện điều chỉnh toán học đối với training error rate để ước lượng test error rate; các approach như vậy sẽ được bàn ở Chapter 6. Trong section này, thay vào đó ta xét một class các method ước lượng test error rate bằng cách giữ lại một subset các training observation khỏi quá trình fitting, rồi áp dụng statistical learning method lên các held-out observation đó.

Trong Sections 5.1.1–5.1.4, để đơn giản ta giả sử mình quan tâm đến regression với một quantitative response. Trong Section 5.1.5, ta xét classification với qualitative response. Như sẽ thấy, các ý tưởng cốt lõi hầu như không thay đổi dù response là quantitative hay qualitative.

5.1.1 The Validation Set Approach

Giả sử ta muốn ước lượng test error gắn với việc fit một statistical learning method cụ thể trên một tập observation. *Validation set approach*, được minh họa trong Figure 5.1, là một strategy rất đơn giản cho nhiệm vụ này. Nó chia ngẫu nhiên tập observation sẵn có thành hai phần: một training set và một validation set, hay *hold-out set*. Model được fit trên training set, sau đó fitted model được dùng để dự đoán response cho các observation trong validation set. Validation set error rate thu được — thường được đánh giá bằng MSE khi response là quantitative — đóng vai trò như một estimate của test error rate.

Ta minh họa validation set approach trên dataset **Auto**. Nhắc lại từ Chapter 3 rằng dường như tồn tại một non-linear relationship giữa **mpg** và **horsepower**, và model dự đoán **mpg** bằng **horsepower** cùng **horsepower**² cho kết quả tốt hơn model chỉ có linear term. Một câu hỏi tự nhiên là liệu cubic fit hoặc higher-order fit có cho kết quả tốt hơn nữa hay không. Trong Chapter 3, ta trả lời câu hỏi này bằng cách xem các p-value gắn với cubic term và các higher-order polynomial term trong linear regression. Nhưng ta cũng có thể trả lời bằng validation method. Ta chia ngẫu nhiên 392 observation thành hai tập: training set gồm 196 data point và validation set gồm 196 observation còn lại. Các validation set error rate thu được khi fit nhiều regression model trên training sample rồi đánh giá performance

*validation set
approach
validation set
hold-out set*



HÌNH 5.1. Minh họa dạng sơ đồ của validation set approach. Một tập gồm n observation được chia ngẫu nhiên thành training set (màu xanh, chứa các observation 7, 22 và 13, cùng những observation khác) và validation set (màu be, chứa observation 91, cùng những observation khác). Statistical learning method được fit trên training set, và performance của nó được đánh giá trên validation set.

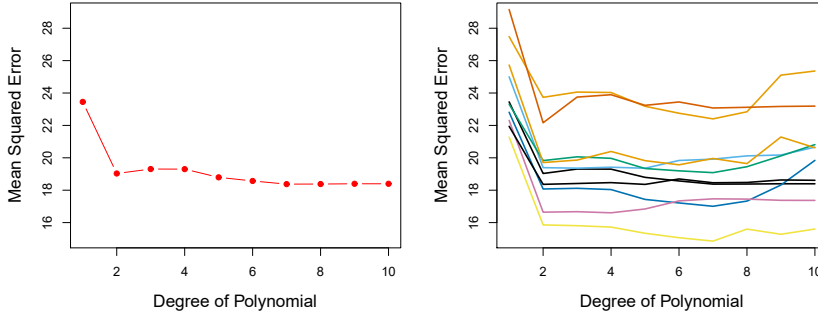
trên validation sample, dùng MSE làm thước đo validation set error, được trình bày ở panel trái của Figure 5.2. Validation set MSE của quadratic fit nhỏ hơn đáng kể so với linear fit. Tuy nhiên, validation set MSE của cubic fit thực ra hơi lớn hơn quadratic fit. Điều này cho thấy việc thêm cubic term vào regression không làm prediction tốt hơn so với chỉ dùng quadratic term.

Nhắc lại rằng để tạo panel trái của Figure 5.2, ta đã chia ngẫu nhiên dataset thành training set và validation set. Nếu lặp lại quá trình chia sample thành hai phần, ta sẽ thu được một estimate hơi khác cho test MSE. Để minh họa, panel phải của Figure 5.2 hiển thị mười validation set MSE curve khác nhau trên dataset Auto, được tạo từ mười cách chia ngẫu nhiên observation thành training set và validation set. Cả mười curve đều cho thấy model có quadratic term có validation set MSE nhỏ hơn rất nhiều so với model chỉ có linear term. Đồng thời, cả mười curve đều cho thấy việc thêm cubic hoặc higher-order polynomial term không mang lại nhiều lợi ích. Tuy nhiên, cần chú ý rằng mỗi curve trong số mười curve lại tạo ra test MSE estimate khác nhau cho từng regression model được xét, và các curve cũng không thống nhất model nào có validation set MSE nhỏ nhất. Dựa trên variability giữa các curve này, điều duy nhất ta có thể kết luận với độ tin cậy đáng kể là linear fit không phù hợp với dữ liệu này.

Validation set approach đơn giản về mặt ý tưởng và dễ implement, nhưng có hai nhược điểm tiềm ẩn:

1. Như panel phải của Figure 5.2 cho thấy, validation estimate của test error rate có thể có variability rất cao, tùy thuộc chính xác observation nào nằm trong training set và observation nào nằm trong validation set.
2. Trong validation approach, chỉ một subset các observation — những observation nằm trong training set thay vì validation set — được dùng để fit model. Vì statistical method thường hoạt động kém hơn khi được training trên ít observation hơn, điều này gợi ý rằng validation set error rate có xu hướng overestimate test error rate của model được fit trên toàn bộ dataset.

Trong các subsection tiếp theo, ta trình bày cross-validation, một refinement của validation set approach nhằm giải quyết hai vấn đề trên.



HÌNH 5.2. Validation set approach được dùng trên dataset *Auto* để ước lượng test error phát sinh khi dự đoán *mpg* bằng các polynomial function của *horsepower*. Left: Validation error estimate cho một lần chia duy nhất thành training và validation dataset. Right: Validation method được lặp lại mười lần, mỗi lần dùng một cách chia ngẫu nhiên khác nhau các observation thành training set và validation set. Hình này minh họa variability trong estimated test MSE do approach này tạo ra.

5.1.2 Leave-One-Out Cross-Validation

Leave-one-out cross-validation (LOOCV) có quan hệ rất gần với validation set approach của Section 5.1.1, nhưng nó cố gắng xử lý các nhược điểm của method đó. Giống validation set approach, LOOCV chia tập observation thành hai phần. Tuy nhiên, thay vì tạo hai subset có size tương đương, một observation (x_1, y_1) được dùng làm validation set, còn các observation còn lại $\{(x_2, y_2), \dots, (x_n, y_n)\}$ tạo thành training set. Statistical learning method được fit trên $n - 1$ training observation, sau đó prediction $\hat{y}_1$ được tạo cho observation bị loại ra bằng giá trị x_1 . Vì (x_1, y_1) không được dùng trong fitting process, $\text{MSE}_1 = (y_1 - \hat{y}_1)^2$ cung cấp một estimate xấp xỉ unbiased cho test error. Nhưng dù MSE_1 là unbiased đối với test error, nó vẫn là một estimate kém vì có variability rất cao do chỉ dựa trên một observation (x_1, y_1) .

leave-one-out
cross-validation

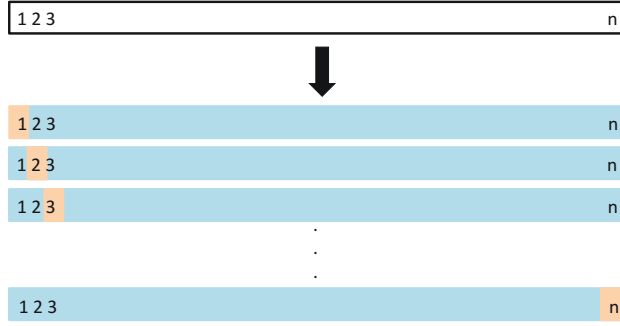
Ta có thể lặp lại procedure bằng cách chọn (x_2, y_2) làm validation data và training statistical learning procedure trên $n - 1$ observation còn lại, tức

$$\{(x_1, y_1), (x_3, y_3), \dots, (x_n, y_n)\}.$$

Sau đó ta tính $\text{MSE}_2 = (y_2 - \hat{y}_2)^2$. Lặp approach này n lần tạo ra n squared error, $\text{MSE}_1, \dots, \text{MSE}_n$. LOOCV estimate cho test MSE là trung bình của n test error estimate này:

$$\text{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^n \text{MSE}_i. \quad (5.1)$$

Sơ đồ LOOCV được minh họa trong Figure 5.3. LOOCV có một vài ưu điểm lớn so với validation set approach. Thứ nhất, bias của nó thấp hơn nhiều. Trong LOOCV, ta lặp lại việc fit statistical learning method bằng các training set chứa $n - 1$ observation, gần bằng số observation trong toàn bộ dataset. Điều này khác với validation set approach, nơi training set thường chỉ bằng khoảng một nửa size của dataset gốc. Vì vậy, LOOCV thường



HÌNH 5.3. Minh họa dạng sơ đồ của LOOCV. Một tập gồm n data point được lặp đi lặp lại việc chia thành training set (màu xanh) chứa tất cả trừ một observation, và validation set chỉ chứa observation đó (màu be). Test error sau đó được ước lượng bằng cách lấy trung bình của n MSE thu được. Training set đầu tiên chứa tất cả trừ observation 1, training set thứ hai chứa tất cả trừ observation 2, và cứ tiếp tục như vậy.

không overestimate test error rate nhiều như validation set approach. Thứ hai, trái với validation approach vốn cho kết quả khác nhau mỗi khi chạy lại do randomness trong cách chia training/validation set, chạy LOOCV nhiều lần luôn tạo ra cùng một kết quả: việc chia training/validation set không có randomness.

Ta đã dùng LOOCV trên dataset **Auto** để ước lượng test set MSE khi fit linear regression model dự đoán **mpg** bằng polynomial function của **horsepower**. Kết quả được trình bày ở panel trái của Figure 5.4.

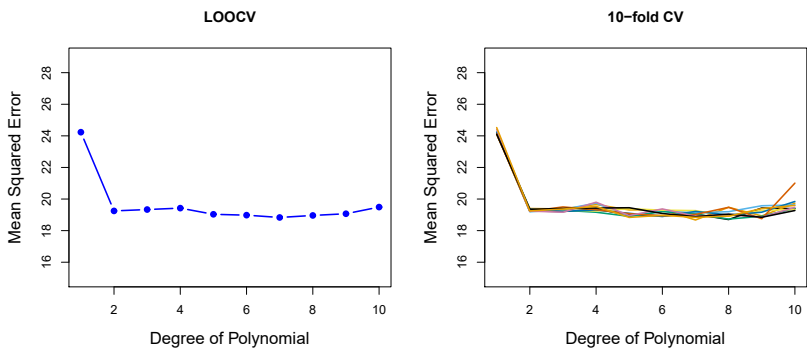
LOOCV có thể tốn kém khi implement vì model phải được fit n lần. Điều này có thể rất chậm nếu n lớn và mỗi individual model mất nhiều thời gian để fit. Với least squares linear hoặc polynomial regression, có một shortcut đáng kinh ngạc khiến chi phí của LOOCV bằng đúng chi phí của một lần fit model. Công thức sau đúng:

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_i} \right)^2. \quad (5.2)$$

Ở đây $\hat{y}_i$ là fitted value thứ i từ original least squares fit, còn h_i là leverage được định nghĩa trong (3.37) ở page 105.¹ Công thức này giống ordinary MSE, ngoại trừ residual thứ i được chia cho $1 - h_i$. Leverage nằm giữa $1/n$ và 1, và phản ánh mức độ một observation ảnh hưởng đến chính fit của nó. Vì vậy residual của các high-leverage point được inflate trong công thức này đúng một lượng cần thiết để equality trên đúng.

LOOCV là một method rất tổng quát và có thể được dùng với hầu như mọi loại predictive modeling. Chẳng hạn, ta có thể dùng nó với logistic regression, linear discriminant analysis, hoặc bất kỳ method nào được thảo luận ở các chapter sau. Tuy nhiên, “magic formula” (5.2) không đúng trong trường hợp tổng quát; khi đó model phải được refit n lần.

¹Trong multiple linear regression, leverage có dạng phức tạp hơn một chút so với (3.37), nhưng (5.2) vẫn đúng.



HÌNH 5.4. Cross-validation được dùng trên dataset *Auto* để ước lượng test error khi dự đoán *mpg* bằng *polynomial function* của *horsepower*. Left: LOOCV error curve. Right: 10-fold CV được chạy chín lần riêng biệt, mỗi lần với một cách chia ngẫu nhiên khác nhau của dữ liệu thành mười phần. Hình cho thấy chín CV error curve hơi khác nhau.

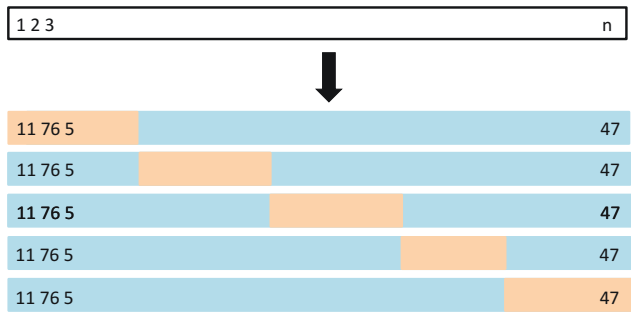
5.1.3 *k*-Fold Cross-Validation

Một alternative cho LOOCV là *k-fold CV*. Approach này chia ngẫu nhiên tập observation thành *k* group, hay *fold*, có size xấp xỉ bằng nhau. Fold đầu tiên được xem là validation set, còn method được fit trên *k* – 1 fold còn lại. Mean squared error MSE_1 sau đó được tính trên các observation trong held-out fold. Procedure này được lặp *k* lần; mỗi lần, một group observation khác được dùng làm validation set. Quá trình này tạo ra *k* estimate của test error, $MSE_1, MSE_2, \dots, MSE_k$. *k*-fold CV estimate được tính bằng cách lấy trung bình:

k-fold CV

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k MSE_i. \tag{5.3}$$

Figure 5.5 minh họa *k*-fold CV approach.

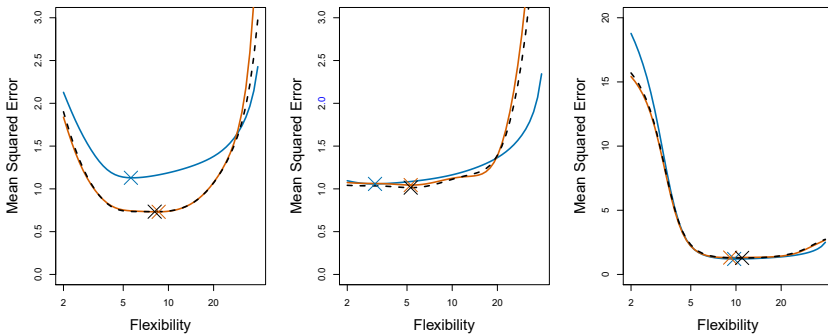


HÌNH 5.5. Minh họa dạng sơ đồ của 5-fold CV. Một tập gồm *n* observation được chia ngẫu nhiên thành năm group không overlap. Mỗi một phần năm lần lượt đóng vai trò validation set (màu be), còn phần còn lại là training set (màu xanh). Test error được ước lượng bằng cách lấy trung bình năm MSE estimate thu được.

Không khó để thấy LOOCV là một trường hợp đặc biệt của k -fold CV khi $k = n$. Trong thực tế, người ta thường dùng k -fold CV với $k = 5$ hoặc $k = 10$. Lợi ích của $k = 5$ hoặc $k = 10$ so với $k = n$ là gì? Lợi ích rõ nhất là computation. LOOCV đòi hỏi fit statistical learning method n lần, có thể rất tốn kém về tính toán (ngoại trừ linear model fit bằng least squares, khi có thể dùng formula (5.2)). Nhưng cross-validation là approach rất tổng quát, có thể áp dụng cho gần như mọi statistical learning method. Một số method có fitting procedure rất nặng, vì thế LOOCV có thể gây vấn đề tính toán, đặc biệt khi n cực lớn. Ngược lại, 10-fold CV chỉ cần fit learning procedure mười lần, khả thi hơn nhiều. Như sẽ thấy ở Section 5.1.4, 5-fold hoặc 10-fold CV còn có những lợi ích khác ngoài computation, liên quan đến bias-variance trade-off.

Panel phải của Figure 5.4 hiển thị chín 10-fold CV estimate khác nhau cho dataset `Auto`, mỗi estimate đến từ một cách chia ngẫu nhiên khác nhau các observation thành mười fold. Hình cho thấy có một mức variability nhất định trong CV estimate do variability của cách chia observation thành mười fold. Tuy vậy, variability này thường thấp hơn rất nhiều so với variability trong test error estimate tạo ra bởi validation set approach (panel phải của Figure 5.2).

Khi làm việc với real data, ta không biết true test MSE nên khó đánh giá accuracy của cross-validation estimate. Nhưng với simulated data, ta có thể tính true test MSE, và nhờ đó đánh giá accuracy của cross-validation result. Trong Figure 5.6, ta plot cross-validation estimate và true test error rate thu được khi áp dụng smoothing spline cho các simulated dataset trong Figures 2.9–2.11 của Chapter 2. True test MSE được vẽ màu xanh. Black dashed line và orange solid line lần lượt biểu diễn LOOCV estimate và 10-fold CV estimate. Trong cả ba plot, hai cross-validation estimate rất giống nhau. Ở panel phải của Figure 5.6, true test MSE và cross-validation curve gần như trùng nhau. Ở panel giữa, hai nhóm curve khá giống nhau tại mức flexibility thấp, còn CV curve overestimate test set MSE khi flexibility cao hơn. Ở panel trái, CV curve có shape tổng thể đúng nhưng underestimate true test MSE.



HÌNH 5.6. True và estimated test MSE cho các simulated dataset trong Figures 2.9 (left), 2.10 (center), và 2.11 (right). True test MSE được vẽ màu xanh, LOOCV estimate là black dashed line, còn 10-fold CV estimate là đường màu cam. Dấu cross chỉ minimum của từng MSE curve.

Khi thực hiện cross-validation, mục tiêu của ta có thể là xác định một

statistical learning procedure cho trước dự kiến sẽ hoạt động tốt đến đâu trên independent data; khi đó actual estimate của test MSE là điều ta quan tâm. Nhưng trong những trường hợp khác, ta chỉ quan tâm vị trí minimum trên estimated test MSE curve. Điều này xảy ra khi ta chạy cross-validation trên nhiều statistical learning method, hoặc trên cùng một method với nhiều mức flexibility khác nhau, để tìm method có test error thấp nhất. Với mục đích này, vị trí minimum của estimated test MSE curve là quan trọng, còn actual value của estimated test MSE thì không. Figure 5.6 cho thấy dù đôi khi underestimate true test MSE, tất cả CV curve vẫn xác định khá gần mức flexibility đúng — tức mức flexibility tương ứng với test MSE nhỏ nhất.

5.1.4 Bias-Variance Trade-Off for k -Fold Cross-Validation

Trong Section 5.1.3, ta đã nói rằng k -fold CV với $k < n$ có lợi thế computation so với LOOCV. Nhưng nếu bỏ qua computation, một lợi thế ít hiển nhiên nhưng có thể quan trọng hơn của k -fold CV là nó thường cho test error rate estimate chính xác hơn LOOCV. Điều này liên quan đến bias-variance trade-off.

Section 5.1.1 đã đề cập rằng validation set approach có thể overestimate test error rate vì training set dùng để fit statistical learning method chỉ chứa một nửa observation của toàn dataset. Theo logic đó, LOOCV sẽ cho estimate gần như unbiased của test error, bởi mỗi training set chứa $n - 1$ observation, gần bằng toàn bộ số observation trong dataset. Còn k -fold CV với, chẳng hạn, $k = 5$ hoặc $k = 10$ sẽ có bias ở mức trung gian vì mỗi training set chứa khoảng $(k - 1)n/k$ observation — ít hơn LOOCV nhưng nhiều hơn đáng kể so với validation set approach. Vì thế, xét riêng khía cạnh giảm bias, LOOCV rõ ràng đáng được ưu tiên hơn k -fold CV.

Tuy nhiên, bias không phải nguồn lo ngại duy nhất của một estimating procedure; ta còn phải xét variance. Hóa ra LOOCV có variance cao hơn k -fold CV với $k < n$. Tại sao? Khi thực hiện LOOCV, ta thực chất đang lấy trung bình output của n fitted model, mỗi model được training trên một tập observation gần như giống hệt nhau; do đó output của chúng có positive correlation rất cao. Ngược lại, với k -fold CV khi $k < n$, ta lấy trung bình output của k fitted model có correlation thấp hơn phần nào, bởi overlap giữa các training set nhỏ hơn. Vì mean của nhiều quantity có correlation cao có variance lớn hơn mean của nhiều quantity không correlation cao đến vậy, test error estimate từ LOOCV thường có variance lớn hơn estimate từ k -fold CV.

Tóm lại, việc chọn k trong k -fold cross-validation kéo theo một bias-variance trade-off. Trong thực tế, dựa trên các cân nhắc này, người ta thường dùng $k = 5$ hoặc $k = 10$, vì empirical evidence cho thấy các giá trị này tạo ra test error rate estimate không bị bias quá cao cũng không có variance quá lớn.

5.1.5 Cross-Validation on Classification Problems

Cho đến đây trong chapter này, ta minh họa cross-validation trong regression setting với quantitative outcome Y , nên đã dùng MSE để định lượng test error. Nhưng cross-validation cũng rất hữu ích trong classification set-

ting khi Y là qualitative. Trong setting này, cross-validation hoạt động y như đã mô tả trước đó, chỉ khác ở chỗ thay vì MSE, ta dùng số observation bị misclassified để định lượng test error. Chẳng hạn, trong classification setting, LOOCV error rate có dạng

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \text{Err}_i, \quad (5.4)$$

trong đó $\text{Err}_i = I(y_i \neq \hat{y}_i)$. k -fold CV error rate và validation set error rate được định nghĩa tương tự.

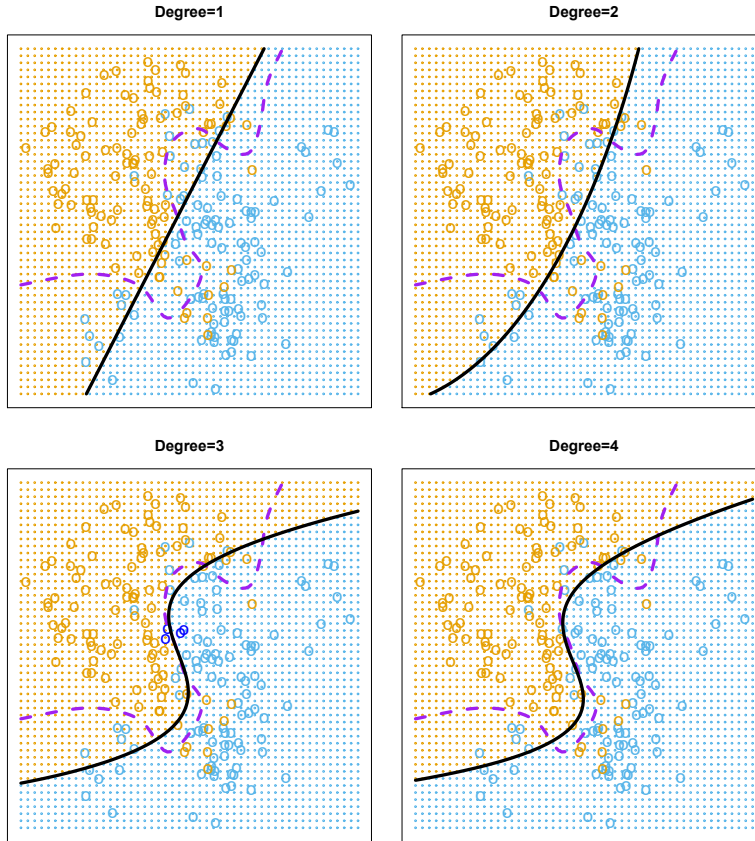
Ví dụ, ta fit nhiều logistic regression model trên two-dimensional classification data trong Figure 2.13. Ở panel trên-trái của Figure 5.7, black solid line biểu diễn estimated decision boundary khi fit standard logistic regression model trên dataset này. Vì đây là simulated data, ta có thể tính true test error rate, bằng 0.201 và do đó lớn hơn đáng kể Bayes error rate 0.133. Rõ ràng logistic regression không đủ flexibility để model Bayes decision boundary trong setting này. Ta có thể mở rộng logistic regression để có non-linear decision boundary bằng polynomial function của predictors, giống cách đã làm trong regression setting ở Section 3.3.2. Chẳng hạn, ta có thể fit quadratic logistic regression model:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \beta_3 X_2 + \beta_4 X_2^2. \quad (5.5)$$

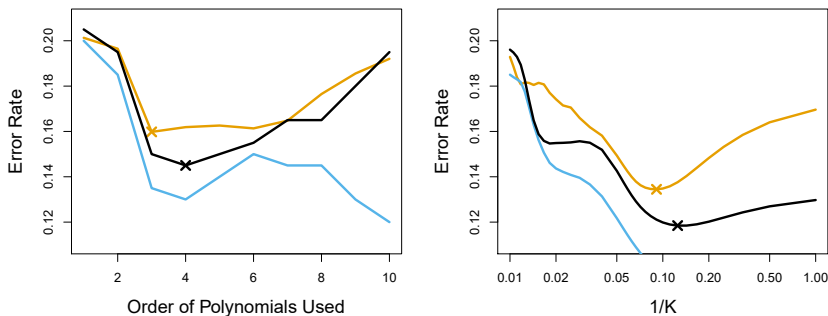
Panel trên-phải của Figure 5.7 cho thấy decision boundary thu được, giờ đã cong. Tuy nhiên, test error rate chỉ cải thiện nhẹ xuống 0.197. Một cải thiện lớn hơn nhiều xuất hiện ở panel dưới-trái, nơi ta fit logistic regression model có cubic polynomial của predictors: test error rate giảm xuống 0.160. Chuyển sang quartic polynomial (panel dưới-phải) khiến test error tăng nhẹ.

Trong thực tế với real data, Bayes decision boundary và test error rate đều không biết trước. Vậy ta chọn thế nào giữa bốn logistic regression model trong Figure 5.7? Ta có thể dùng cross-validation. Panel trái của Figure 5.8 hiển thị bằng màu đen 10-fold CV error rate khi fit mười logistic regression model, dùng polynomial function của predictors đến order 10. True test error được vẽ màu nâu, training error màu xanh. Như đã thấy, training error có xu hướng giảm khi flexibility của fit tăng. Trái lại, test error có U-shape đặc trưng. 10-fold CV error rate xấp xỉ khá tốt test error rate. Dù hơi underestimate error rate, nó đạt minimum khi dùng fourth-order polynomial, rất gần minimum của test curve xảy ra với third-order polynomial. Trên thực tế, fourth-order polynomial có lẽ vẫn cho test set performance tốt vì true test error rate gần như tương đương cho polynomial order 3, 4, 5 và 6.

Panel phải của Figure 5.8 hiển thị ba curve tương tự khi dùng KNN classification như một function của K (ở đây K là số neighbor của KNN classifier, không phải số CV fold). Một lần nữa, training error rate giảm khi method linh hoạt hơn, nên training error rate không thể dùng để chọn K tối ưu. Dù cross-validation error curve hơi underestimate test error rate, minimum của nó nằm rất gần best value của K .



HÌNH 5.7. Logistic regression fit trên two-dimensional classification data trong Figure 2.13. Bayes decision boundary được biểu diễn bằng purple dashed line. Estimated decision boundary từ linear, quadratic, cubic và quartic (degree 1–4) logistic regression được vẽ màu đen. Test error rate của bốn logistic regression fit lần lượt là 0.201, 0.197, 0.160 và 0.162, còn Bayes error rate là 0.133.



HÌNH 5.8. Test error (nâu), training error (xanh), và 10-fold CV error (đen) trên two-dimensional classification data trong Figure 5.7. Left: Logistic regression dùng polynomial function của predictors; polynomial order được hiển thị trên trục x . Right: KNN classifier với nhiều giá trị K , là số neighbor được dùng trong KNN classifier.

5.2 The Bootstrap

Bootstrap là một statistical tool cực kỳ mạnh và có phạm vi áp dụng rộng, có thể dùng để định lượng uncertainty gắn với một estimator hoặc statistical learning method cho trước. Chẳng hạn, bootstrap có thể được dùng để ước lượng standard error của các coefficient từ một linear regression fit. Riêng trong linear regression, điều này không đặc biệt hữu ích vì ở Chapter 3 ta đã thấy các statistical software chuẩn tự động output các standard error này. Tuy nhiên, sức mạnh của bootstrap nằm ở chỗ nó có thể áp dụng rất dễ dàng cho rất nhiều statistical learning method, bao gồm cả những method mà nếu không dùng bootstrap thì variability rất khó định lượng và software cũng không tự động output.

bootstrap

Trong section này, ta minh họa bootstrap bằng một toy example, nơi mục tiêu là xác định investment allocation tốt nhất dưới một model đơn giản. Trong Section 5.3, ta sẽ dùng bootstrap để đánh giá variability gắn với regression coefficient trong một linear model fit.

Giả sử ta muốn đầu tư một số tiền cố định vào hai financial asset tạo ra return lần lượt là X và Y , trong đó X và Y là random quantity. Ta đầu tư fraction α của số tiền vào X , và phần còn lại $1 - \alpha$ vào Y . Vì return của hai asset có variability, ta muốn chọn α sao cho total risk, hay variance, của investment nhỏ nhất. Nói cách khác, ta muốn minimize $\text{Var}(\alpha X + (1 - \alpha)Y)$. Có thể chứng minh rằng giá trị minimize risk là

$$\alpha = \frac{\sigma_Y^2 - \sigma_{XY}}{\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}}, \quad (5.6)$$

trong đó $\sigma_X^2 = \text{Var}(X)$, $\sigma_Y^2 = \text{Var}(Y)$, và $\sigma_{XY} = \text{Cov}(X, Y)$.

Trong thực tế, các quantity σ_X^2 , σ_Y^2 , và σ_{XY} là unknown. Ta có thể tính các estimate $\hat{\sigma}_X^2$, $\hat{\sigma}_Y^2$, và $\hat{\sigma}_{XY}$ từ một dataset chứa các measurement trong quá khứ của X và Y . Khi đó, giá trị α làm minimize variance của investment có thể được estimate bằng

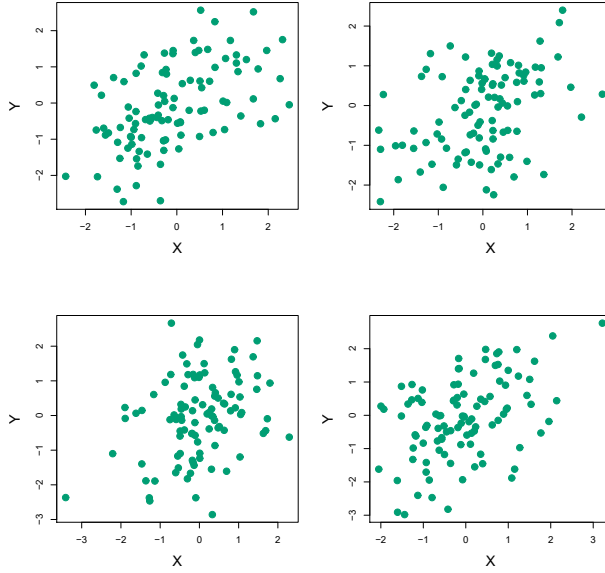
$$\hat{\alpha} = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}. \quad (5.7)$$

Figure 5.9 minh họa approach này trên simulated data. Trong mỗi panel, ta simulate 100 pair return cho hai investment X và Y . Ta dùng các return này để estimate σ_X^2 , σ_Y^2 , và σ_{XY} , rồi thay vào (5.7) để thu được estimate của α . Giá trị $\hat{\alpha}$ từ bốn simulated dataset nằm trong khoảng 0.532 đến 0.657.

Ta tự nhiên muốn định lượng accuracy của estimate α . Để estimate standard deviation của $\hat{\alpha}$, ta lặp lại quá trình simulate 100 paired observation của X và Y và estimate α bằng (5.7) tổng cộng 1,000 lần. Qua đó ta thu được 1,000 estimate $\hat{\alpha}_1, \hat{\alpha}_2, \dots, \hat{\alpha}_{1,000}$. Panel trái của Figure 5.10 hiển thị histogram của các estimate này. Trong simulation, parameter được đặt là $\sigma_X^2 = 1$, $\sigma_Y^2 = 1.25$, và $\sigma_{XY} = 0.5$, nên true value của α là 0.6. Giá trị này được biểu diễn bằng solid vertical line trên histogram.

Mean của toàn bộ 1,000 estimate của α là

$$\bar{\alpha} = \frac{1}{1000} \sum_{r=1}^{1000} \hat{\alpha}_r = 0.5996,$$



HÌNH 5.9. Mỗi panel hiển thị 100 simulated return cho investments X và Y . Từ trái sang phải, từ trên xuống dưới, các estimate α thu được lần lượt là 0.576, 0.532, 0.657, và 0.651.

rất gần $\alpha = 0.6$, và standard deviation của các estimate là

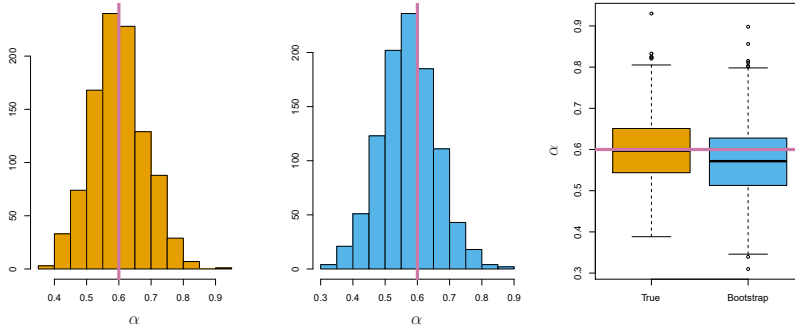
$$\sqrt{\frac{1}{1000-1} \sum_{r=1}^{1000} (\hat{\alpha}_r - \bar{\alpha})^2} = 0.083.$$

Điều này cho ta một hình dung rất tốt về accuracy của $\hat{\alpha}$: $SE(\hat{\alpha}) \approx 0.083$. Nói gần đúng, với một random sample từ population, ta kỳ vọng $\hat{\alpha}$ khác α khoảng 0.08, trung bình.

Tuy nhiên, trong thực tế procedure vừa dùng để estimate $SE(\hat{\alpha})$ không thể áp dụng trực tiếp, vì với real data ta không thể tạo các sample mới từ original population. Bootstrap giải quyết vấn đề này bằng cách dùng computer để mô phỏng quá trình tạo sample set mới, nhờ đó estimate variability của $\hat{\alpha}$ mà không cần thu thập thêm sample. Thay vì liên tục lấy các independent dataset từ population, ta tạo các dataset khác nhau bằng cách liên tục sample observation từ original dataset.

Approach này được minh họa trong Figure 5.11 trên một dataset đơn giản, ký hiệu Z , chỉ có $n = 3$ observation. Ta chọn ngẫu nhiên n observation từ dataset để tạo bootstrap dataset Z^{*1} . Sampling được thực hiện *with replacement*, nghĩa là cùng một observation có thể xuất hiện nhiều hơn một lần trong bootstrap dataset. Trong ví dụ này, Z^{*1} chứa observation thứ ba hai lần, observation thứ nhất một lần, và không có observation thứ hai. Lưu ý rằng nếu một observation nằm trong Z^{*1} thì cả giá trị X và Y của observation đó đều được đưa vào. Từ Z^{*1} ta tạo bootstrap estimate mới của α , ký hiệu $\hat{\alpha}^{*1}$. Procedure được lặp B lần với B đủ lớn, tạo ra B bootstrap dataset $Z^{*1}, Z^{*2}, \dots, Z^{*B}$ và B estimate tương ứng $\hat{\alpha}^{*1}, \hat{\alpha}^{*2}, \dots, \hat{\alpha}^{*B}$.

with replacement



HÌNH 5.10. Left: Histogram của các estimate α thu được bằng cách tạo 1,000 simulated dataset từ true population. Center: Histogram của các estimate α thu được từ 1,000 bootstrap sample của một dataset duy nhất. Right: Các estimate α ở panel trái và giữa được hiển thị dưới dạng boxplot. Trong mỗi panel, đường màu hồng chỉ true value của α .

Ta có thể tính standard error của các bootstrap estimate bằng formula

$$SE_B(\hat{\alpha}) = \sqrt{\frac{1}{B-1} \sum_{r=1}^B \left(\hat{\alpha}^{*r} - \frac{1}{B} \sum_{r'=1}^B \hat{\alpha}^{*r'} \right)^2}. \quad (5.8)$$

Quantity này đóng vai trò estimate cho standard error của $\hat{\alpha}$ được tính từ original dataset.

Bootstrap approach được minh họa ở panel giữa của Figure 5.10, hiển thị histogram của 1,000 bootstrap estimate của α , mỗi estimate được tính từ một bootstrap dataset khác nhau. Panel này được dựng chỉ từ một dataset duy nhất, nên có thể tạo từ real data. Histogram trông rất giống panel trái, nơi hiển thị idealized histogram của các estimate α thu được bằng cách generate 1,000 simulated dataset từ true population. Đặc biệt, bootstrap estimate $SE(\hat{\alpha})$ từ (5.8) bằng 0.087, rất gần estimate 0.083 thu được từ 1,000 simulated dataset. Panel phải biểu diễn lại thông tin ở hai panel trái và giữa bằng boxplot. Một lần nữa, spread của hai boxplot rất giống nhau, cho thấy bootstrap có thể estimate variability gần với $\hat{\alpha}$ một cách hiệu quả.

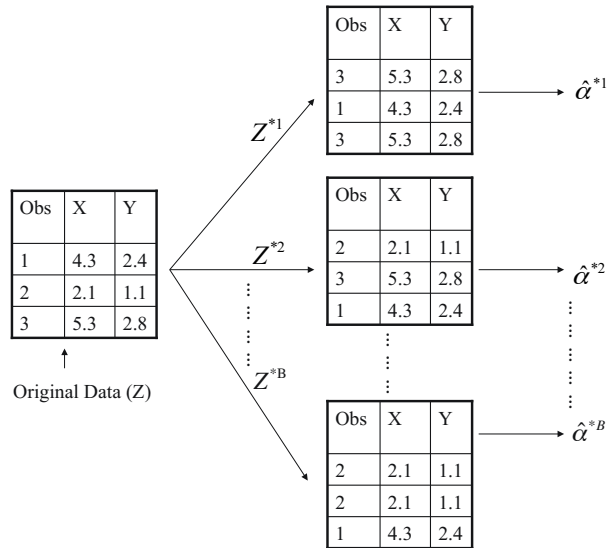
5.3 Lab: Cross-Validation and the Bootstrap

Trong lab này, ta khám phá các resampling technique đã được trình bày trong chapter. Một số command trong lab có thể mất một lúc để chạy trên computer của bạn.

Ta lại bắt đầu bằng cách đặt phần lớn các import ở top level.

In [1]:

```
import numpy as np
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
                          summarize,
                          poly)
from sklearn.model_selection import train_test_split
```



HÌNH 5.11. Minh họa bằng đồ thị của bootstrap approach trên một sample nhỏ gồm $n = 3$ observation. Mỗi bootstrap dataset chứa n observation được sample with replacement từ original dataset. Mỗi bootstrap dataset được dùng để tạo một estimate của α .

Lab này cần thêm một số import mới.

```
In [2]: from functools import partial
from sklearn.model_selection import \
    (cross_validate,
     KFold,
     ShuffleSplit)
from sklearn.base import clone
from ISLP.models import sklearn_sm
```

5.3.1 The Validation Set Approach

Ta dùng validation set approach để estimate test error rate khi fit nhiều linear model khác nhau trên dataset Auto.

Ta dùng function `train_test_split()` để chia dữ liệu thành training set và validation set. Vì có 392 observation, ta chia thành hai tập bằng nhau size 196 bằng argument `test_size=196`. Nói chung nên đặt random seed khi thực hiện những operation có yếu tố randomness như thế này, để kết quả có thể được reproduce chính xác ở thời điểm sau. Ta đặt random seed của splitter bằng argument `random_state=0`.

`train_test_split()`

```
In [3]: Auto = load_data('Auto')
Auto_train, Auto_valid = train_test_split(Auto,
                                          test_size=196,
                                          random_state=0)
```

Bây giờ ta có thể fit linear regression chỉ bằng các observation thuộc training set `Auto_train`.

```
In [4]: hp_mm = MS(['horsepower'])
X_train = hp_mm.fit_transform(Auto_train)
y_train = Auto_train['mpg']
model = sm.OLS(y_train, X_train)
results = model.fit()
```

Tiếp theo, ta dùng method `predict()` của `results`, đánh giá trên model matrix của model này được tạo từ validation dataset. Đồng thời ta tính validation MSE của model.

```
In [5]: X_valid = hp_mm.transform(Auto_valid)
y_valid = Auto_valid['mpg']
valid_pred = results.predict(X_valid)
np.mean((y_valid - valid_pred)**2)
```

```
Out[5]: 23.6166
```

Vì vậy estimate của validation MSE cho linear regression fit là 23.62.

Ta cũng có thể estimate validation error cho higher-degree polynomial regression. Trước hết ta định nghĩa function `evalMSE()` nhận vào một model string cùng training set và test set, rồi trả về MSE trên test set.

```
In [6]: def evalMSE(terms,
                response,
                train,
                test):

    mm = MS(terms)
    X_train = mm.fit_transform(train)
    y_train = train[response]

    X_test = mm.transform(test)
    y_test = test[response]

    results = sm.OLS(y_train, X_train).fit()
    test_pred = results.predict(X_test)

    return np.mean((y_test - test_pred)**2)
```

Hãy dùng function này để estimate validation MSE cho linear, quadratic và cubic fit. Ở đây ta dùng function `enumerate()`, function này cung cấp cả value lẫn index của object khi ta iterate qua một for loop.

enumerate()

```
In [7]: MSE = np.zeros(3)
for idx, degree in enumerate(range(1, 4)):
    MSE[idx] = evalMSE([poly('horsepower', degree)],
                        'mpg',
                        Auto_train,
                        Auto_valid)

MSE
```

```
Out[7]: array([23.62, 18.76, 18.80])
```

Ba error rate lần lượt là 23.62, 18.76 và 18.80. Nếu chọn một cách chia training/validation khác, ta kỳ vọng validation set error cũng sẽ khác phần nào.

```
In [8]: Auto_train, Auto_valid = train_test_split(Auto,
                                                test_size=196,
                                                random_state=3)

MSE = np.zeros(3)
for idx, degree in enumerate(range(1, 4)):
    MSE[idx] = evalMSE([poly('horsepower', degree)],
                      'mpg',
                      Auto_train,
                      Auto_valid)

MSE
```

```
Out[8]: array([20.76, 16.95, 16.97])
```

Với cách chia này, validation set error rate của model có linear, quadratic và cubic term lần lượt là 20.76, 16.95 và 16.97. Kết quả phù hợp với những gì ta đã thấy trước đó: model dự đoán mpg bằng quadratic function của horsepower hoạt động tốt hơn model chỉ có linear function của horsepower, và không có evidence cho thấy cubic function cải thiện thêm.

5.3.2 Cross-Validation

Về lý thuyết, cross-validation estimate có thể được tính cho bất kỳ generalized linear model nào. Trong thực tế, cách đơn giản nhất để thực hiện cross-validation trong Python là dùng **sklearn**, thư viện có interface hay API khác với **statsmodels**, là code mà ta đã dùng để fit GLM.

Đây là một vấn đề data scientist thường gặp: “Tôi có một function làm task A, và cần đưa nó vào một thứ thực hiện task B, để có thể tính $B(A(D))$, với D là data.” Khi A và B không tự nhiên tương tác với nhau, ta cần một *wrapper*. Trong package ISLP, ta cung cấp wrapper **sklearn_sm()** để dễ dàng dùng các cross-validation tool của **sklearn** với model fit bằng **statsmodels**.

Class **sklearn_sm()** nhận argument đầu tiên là một model từ **statsmodels**. Nó có thể nhận thêm hai optional argument: **model_str**, dùng để chỉ định formula, và **model_args**, là dictionary chứa các additional argument dùng khi fit model. Ví dụ, để fit logistic regression model ta phải chỉ định family argument; giá trị được truyền dưới dạng sau:

wrapper
sklearn_sm()

```
model_args={'family':sm.families.Binomial()}
```

Dưới đây là wrapper đang hoạt động:

```
In [9]: hp_model = sklearn_sm(sm.OLS,
                             MS(['horsepower']))
X, Y = Auto.drop(columns=['mpg'], Auto['mpg'])
cv_results = cross_validate(hp_model,
                           X,
                           Y,
                           cv=Auto.shape[0])
cv_err = np.mean(cv_results['test_score'])
cv_err
```

```
Out[9]: 24.2315
```

Các argument của **cross_validate()** gồm: một object có các method

`fit()`, `predict()` và `score()` phù hợp, một array feature X , và response Y . Ta còn truyền argument `cv`; nếu chỉ định một integer K , function sẽ thực hiện K -fold cross-validation. Ở đây ta truyền tổng số observation, nên thu được leave-one-out cross-validation (LOOCV). Function `cross_validate()` tạo ra một dictionary với nhiều component; ở đây ta chỉ cần cross-validated test score (MSE), estimate bằng 24.23.

`cross_validate()`

Ta có thể lặp procedure này cho các polynomial fit ngày càng phức tạp. Để tự động hóa, ta lại dùng for loop lần lượt fit polynomial regression degree 1 đến 5, tính cross-validation error tương ứng và lưu vào phần tử thứ i của vector `cv_error`. Variable `d` trong loop là degree của polynomial. Trước hết ta initialize vector; command này có thể mất vài giây.

In [10]:

```
cv_error = np.zeros(5)
H = np.array(Auto['horsepower'])
M = sklearn_sm(sm.OLS)
for i, d in enumerate(range(1,6)):
    X = np.power.outer(H, np.arange(d+1))
    M_CV = cross_validate(M,
                          X,
                          Y,
                          cv=Auto.shape[0])
    cv_error[i] = np.mean(M_CV['test_score'])
cv_error
```

Out[10]:

```
array([24.2315, 19.2482, 19.3350, 19.4244, 19.0332])
```

Như Figure 5.4, estimated test MSE giảm mạnh từ linear fit sang quadratic fit, nhưng sau đó không có cải thiện rõ ràng khi dùng higher-degree polynomial.

Ở trên ta đã giới thiệu method `outer()` của function `np.power()`. Method `outer()` được áp dụng cho một operation có hai argument, như `add()`, `min()` hay `power()`. Nó nhận hai array và tạo một array lớn hơn, trong đó operation được áp dụng cho từng pair element của hai array.

`.outer()`
`np.power()`

In [11]:

```
A = np.array([3, 5, 9])
B = np.array([2, 4])
np.add.outer(A, B)
```

Out[11]:

```
array([[ 5,  7],
       [ 7,  9],
       [11, 13]])
```

Trong ví dụ CV ở trên, ta dùng $K = n$, nhưng tất nhiên cũng có thể dùng $K < n$. Code rất giống phần trên và nhanh hơn đáng kể. Ở đây ta dùng `KFold()` để chia dữ liệu thành $K = 10$ random group. Ta dùng `random_state` để đặt random seed và initialize vector `cv_error`, nơi lưu CV error ứng với polynomial fit degree 1 đến 5.

`KFold()`

In [12]:

```
cv_error = np.zeros(5)
cv = KFold(n_splits=10,
          shuffle=True,
          random_state=0) # use same splits for each degree
for i, d in enumerate(range(1,6)):
    X = np.power.outer(H, np.arange(d+1))
    M_CV = cross_validate(M,
                          X,
```

```

        Y,
        cv=cv)
    cv_error[i] = np.mean(M_CV['test_score'])
cv_error

```

Out[12]: array([24.2077, 19.1853, 19.2763, 19.4785, 19.1372])

Computation time ngắn hơn nhiều so với LOOCV. Về nguyên tắc, với least squares linear model, LOOCV phải nhanh hơn K -fold CV nhờ formula (5.2); tuy nhiên generic function `cross_validate()` không sử dụng formula này. Ta vẫn thấy rất ít evidence rằng cubic hoặc higher-degree polynomial term làm test error thấp hơn quadratic fit.

Function `cross_validate()` linh hoạt và có thể nhận nhiều splitting mechanism khác nhau làm argument. Chẳng hạn, function `ShuffleSplit()` có thể implement validation set approach dễ dàng như K -fold cross-validation.

ShuffleSplit()

```

In [13]: validation = ShuffleSplit(n_splits=1,
                                test_size=196,
                                random_state=0)
results = cross_validate(hp_model,
                          Auto.drop(['mpg'], axis=1),
                          Auto['mpg'],
                          cv=validation);
results['test_score']

```

Out[13]: array([23.6166])

Ta có thể estimate variability trong test error bằng cách chạy:

```

In [14]: validation = ShuffleSplit(n_splits=10,
                                test_size=196,
                                random_state=0)
results = cross_validate(hp_model,
                          Auto.drop(['mpg'], axis=1),
                          Auto['mpg'],
                          cv=validation)
results['test_score'].mean(), results['test_score'].std()

```

Out[14]: (23.8022, 1.4218)

Lưu ý standard deviation này không phải estimate hợp lệ của sampling variability của mean test score hay của individual score, vì các training sample được chọn ngẫu nhiên overlap nhau và do đó tạo correlation. Tuy vậy, nó cho ta hình dung về Monte Carlo variation phát sinh khi chọn các random fold khác nhau.

5.3.3 The Bootstrap

Ta minh họa bootstrap bằng ví dụ đơn giản của Section 5.2, đồng thời qua một ví dụ estimate accuracy của linear regression model trên dataset `Auto`. *Estimating the Accuracy of a Statistic of Interest*

Một lợi thế lớn của bootstrap là nó có thể áp dụng trong gần như mọi tình huống mà không cần calculation toán học phức tạp. Dù Python có

những bootstrap implementation, việc dùng bootstrap để estimate standard error đơn giản đến mức ta tự viết function cho trường hợp data được lưu trong dataframe.

Ta bắt đầu bằng ví dụ đơn giản. Dataset `Portfolio` trong package `ISLP` được mô tả ở Section 5.2. Mục tiêu là estimate sampling variance của parameter α trong formula (5.7). Ta tạo function `alpha_func()`, nhận input là dataframe `D` được giả định có các column `X`, `Y`, cùng vector `idx` chỉ các observation cần dùng để estimate α . Function output estimate của α từ các observation được chọn.

In [15]:

```
Portfolio = load_data('Portfolio')
def alpha_func(D, idx):
    cov_ = np.cov(D[['X', 'Y']].loc[idx], rowvar=False)
    return ((cov_[1,1] - cov_[0,1]) /
            (cov_[0,0]+cov_[1,1]-2*cov_[0,1]))
```

Function này trả về estimate của α bằng cách áp dụng minimum variance formula (5.7) lên các observation được index bởi `idx`. Chẳng hạn, command sau estimate α bằng toàn bộ 100 observation.

In [16]:

```
alpha_func(Portfolio, range(100))
```

Out[16]:

0.5758

Tiếp theo ta chọn ngẫu nhiên 100 observation từ `range(100)`, with replacement. Điều này tương đương tạo một bootstrap dataset mới rồi tính lại $\hat{\alpha}$ trên dataset đó.

In [17]:

```
rng = np.random.default_rng(0)
alpha_func(Portfolio,
            rng.choice(100,
                       100,
                       replace=True))
```

Out[17]:

0.6074

Process này có thể được generalized thành function đơn giản `boot_SE()` để tính bootstrap standard error cho arbitrary function chỉ nhận dataframe làm argument.

In [18]:

```
def boot_SE(func,
            D,
            n=None,
            B=1000,
            seed=0):
    rng = np.random.default_rng(seed)
    first_, second_ = 0, 0
    n = n or D.shape[0]
    for _ in range(B):
        idx = rng.choice(D.index,
                        n,
                        replace=True)
        value = func(D, idx)
        first_ += value
        second_ += value**2
    return np.sqrt(second_ / B - (first_ / B)**2)
```

Lưu ý việc dùng `_` làm loop variable trong `for _ in range(B)`. Cách viết này thường được dùng khi value của counter không quan trọng và chỉ cần bảo đảm loop chạy đúng B lần.

Hãy dùng function này để đánh giá accuracy của estimate α với $B = 1,000$ bootstrap replication.

In [19]:

```
alpha_SE = boot_SE(alpha_func,
                    Portfolio,
                    B=1000,
                    seed=0)

alpha_SE
```

Out[19]:

0.0912

Output cuối cho thấy bootstrap estimate của $SE(\hat{\alpha})$ là 0.0912.

Estimating the Accuracy of a Linear Regression Model

Bootstrap có thể dùng để đánh giá variability của coefficient estimate và prediction từ một statistical learning method. Ở đây ta dùng bootstrap để đánh giá variability của estimate β_0 và β_1 , tức intercept và slope của linear regression model dùng **horsepower** để dự đoán **mpg** trên dataset **Auto**. Ta sẽ so sánh estimate thu được từ bootstrap với estimate từ các formula cho $SE(\hat{\beta}_0)$ và $SE(\hat{\beta}_1)$ ở Section 3.1.2.

Để dùng function `boot_SE()`, ta phải viết một function nhận dataframe `D` và indices `idx` làm hai argument duy nhất. Nhưng ở đây ta muốn bootstrap một regression model cụ thể được chỉ định bởi model formula và data. Ta thực hiện điều đó qua vài bước đơn giản.

Trước hết ta viết generic function `boot_OLS()` để bootstrap regression model; function nhận một formula xác định regression tương ứng. Ta dùng function `clone()` để tạo copy của formula có thể được refit trên dataframe mới. Điều này có nghĩa các derived feature như những feature định nghĩa bằng `poly()` cũng sẽ được refit trên resampled dataframe.

clone()

In [20]:

```
def boot_OLS(model_matrix, response, D, idx):
    D_ = D.loc[idx]
    Y_ = D_[response]
    X_ = clone(model_matrix).fit_transform(D_)
    return sm.OLS(Y_, X_).fit().params
```

Đây vẫn chưa chính xác là thứ ta cần làm argument đầu tiên của `boot_SE()`. Hai argument đầu chỉ định model sẽ không thay đổi trong bootstrap process, nên ta muốn freeze chúng. Function `partial()` từ module `functools` làm đúng việc này: nó nhận một function làm argument rồi freeze một số argument của function đó, bắt đầu từ bên trái. Ta dùng nó để freeze hai model-formula argument đầu của `boot_OLS()`.

partial()

In [21]:

```
hp_func = partial(boot_OLS, MS(['horsepower']), 'mpg')
```

Gỡ `hp_func`? sẽ cho thấy function có hai argument `D` và `idx`: đây là version của `boot_OLS()` trong đó hai argument đầu đã được freeze, và vì thế rất phù hợp làm argument đầu cho `boot_SE()`.

Function `hp_func()` giờ có thể dùng để tạo bootstrap estimate cho intercept và slope term bằng cách sample ngẫu nhiên các observation with

replacement. Ta trước hết minh họa utility của nó trên 10 bootstrap sample.

```
In [22]: rng = np.random.default_rng(0)
          np.array([hp_func(Auto,
                           rng.choice(392,
                                       392,
                                       replace=True)) for _ in range(10)])
```

```
Out[22]: array([[39.8806, -0.1568],
                [38.733 , -0.147 ],
                [38.3173, -0.1444],
                [39.9145, -0.1578],
                [39.4335, -0.1507],
                [40.3663, -0.1591],
                [39.6233, -0.1545],
                [39.0581, -0.1495],
                [38.6669, -0.1452],
                [39.6428, -0.1556]])
```

Tiếp theo, ta dùng `boot_SE()` để tính standard error của 1,000 bootstrap estimate cho intercept và slope term.

```
In [23]: hp_se = boot_SE(hp_func,
                          Auto,
                          B=1000,
                          seed=10)

          hp_se
```

```
Out[23]: intercept    0.8488
          horsepower    0.0074
          dtype: float64
```

Điều này cho biết bootstrap estimate của $SE(\hat{\beta}_0)$ là 0.85 và của $SE(\hat{\beta}_1)$ là 0.0074. Như đã thảo luận trong Section 3.1.2, có thể dùng các standard formula để tính standard error cho regression coefficient trong linear model. Ta lấy chúng bằng function `summarize()`.

```
In [24]: hp_model.fit(Auto, Auto['mpg'])
          model_se = summarize(hp_model.results_)['std err']
          model_se
```

```
Out[24]: intercept    0.717
          horsepower    0.006
          Name: std err, dtype: float64
```

Standard error estimate cho $\hat{\beta}_0$ và $\hat{\beta}_1$ từ formula ở Section 3.1.2 lần lượt là 0.717 cho intercept và 0.006 cho slope. Khả thú vị là chúng khác phần nào với estimate từ bootstrap. Điều này có phải cho thấy bootstrap có vấn đề? Thực ra nó gợi ý điều ngược lại. Nhắc lại rằng các standard formula trong Equation 3.8 ở page 75 dựa trên một số assumption. Chẳng hạn, chúng phụ thuộc vào unknown parameter σ^2 , noise variance, rồi ta estimate σ^2 bằng RSS. Dù formula cho standard error không đòi linear model phải đúng, estimate của σ^2 thì có. Figure 3.8 ở page 99 cho thấy data có non-linear relationship, nên residual từ linear fit sẽ bị inflate, kéo theo $\hat{\sigma}^2$ cũng bị inflate. Thứ hai, standard formula giả định — hơi thiếu thực tế — rằng x_i

là fixed và toàn bộ variability đến từ error ϵ_i . Bootstrap không dựa trên các assumption này, vì vậy có khả năng nó cung cấp estimate chính xác hơn cho standard error của $\hat{\beta}_0$ và $\hat{\beta}_1$ so với kết quả từ `sm.OLS`.

Dưới đây ta tính bootstrap standard error estimate và standard linear regression estimate khi fit quadratic model. Vì model này fit data khá tốt (Figure 3.8), giờ đây bootstrap estimate và standard estimate của $SE(\hat{\beta}_0)$, $SE(\hat{\beta}_1)$, $SE(\hat{\beta}_2)$ tương ứng tốt hơn.

```
In [25]: quad_model = MS([poly('horsepower', 2, raw=True)])
quad_func = partial(boot_OLS,
                    quad_model,
                    'mpg')
boot_SE(quad_func, Auto, B=1000)
```

```
Out[25]: intercept                2.067840
poly(horsepower, 2, raw=True)[0]  0.033019
poly(horsepower, 2, raw=True)[1]  0.000120
dtype: float64
```

Ta so sánh kết quả với standard error tính bằng `sm.OLS()`.

```
In [26]: M = sm.OLS(Auto['mpg'],
                    quad_model.fit_transform(Auto))
summarize(M.fit())['std err']
```

```
Out[26]: intercept                1.800
poly(horsepower, 2, raw=True)[0]  0.031
poly(horsepower, 2, raw=True)[1]  0.000
Name: std err, dtype: float64
```

5.4 Exercises

Conceptual

- Dùng các property thống kê cơ bản của variance, cùng single-variable calculus, hãy derive (5.6). Nói cách khác, chứng minh rằng α trong (5.6) thực sự minimize $\text{Var}(\alpha X + (1 - \alpha)Y)$.
- Bây giờ ta sẽ derive probability rằng một observation cho trước thuộc một bootstrap sample. Giả sử ta lấy một bootstrap sample từ một tập gồm n observation.
 - Probability rằng bootstrap observation đầu tiên *không phải* observation thứ j của original sample là bao nhiêu? Hãy giải thích.
 - Probability rằng bootstrap observation thứ hai không phải observation thứ j của original sample là bao nhiêu?
 - Hãy lập luận rằng probability observation thứ j không nằm trong bootstrap sample là $(1 - 1/n)^n$.
 - Khi $n = 5$, probability observation thứ j nằm trong bootstrap sample là bao nhiêu?
 - Khi $n = 100$, probability observation thứ j nằm trong bootstrap sample là bao nhiêu?

- (f) Khi $n = 10,000$, probability observation thứ j nằm trong bootstrap sample là bao nhiêu?
- (g) Tạo một plot hiển thị, với mỗi integer n từ 1 đến 100,000, probability observation thứ j nằm trong bootstrap sample. Hãy nhận xét điều bạn quan sát được.
- (h) Bây giờ ta khảo sát bằng numerical experiment probability rằng một bootstrap sample size $n = 100$ chứa observation thứ j , với $j = 4$. Trước hết tạo array **store** bằng function `np.empty()`, các value trong array sẽ được overwrite sau đó. Tiếp theo, liên tục tạo bootstrap sample, và mỗi lần ghi lại liệu observation thứ n có nằm trong bootstrap sample hay không.

`np.empty()`

```
rng = np.random.default_rng(10)
store = np.empty(10000)
for i in range(10000):
    store[i] = np.sum(rng.choice(100, replace=True) == 4)
    > 0
np.mean(store)
```

Hãy nhận xét kết quả thu được.

3. Bây giờ ta review k -fold cross-validation.
 - (a) Giải thích cách implement k -fold cross-validation.
 - (b) Ưu điểm và nhược điểm của k -fold cross-validation so với:
 - i. Validation set approach?
 - ii. LOOCV?
4. Giả sử ta dùng một statistical learning method để prediction response Y tại một giá trị cụ thể của predictor X . Hãy mô tả cẩn thận cách ta có thể estimate standard deviation của prediction đó.

Applied

5. Trong Chapter 4, ta đã dùng logistic regression để dự đoán probability of default bằng **income** và **balance** trên dataset **Default**. Bây giờ ta estimate test error của logistic regression model này bằng validation set approach. Đừng quên đặt random seed trước khi bắt đầu analysis.
 - (a) Fit logistic regression model dùng **income** và **balance** để dự đoán **default**.
 - (b) Dùng validation set approach để estimate test error của model. Để làm điều này, bạn phải thực hiện các bước sau:
 - i. Chia sample thành training set và validation set.
 - ii. Fit multiple logistic regression model chỉ bằng các training observation.
 - iii. Tạo prediction về default status cho từng individual trong validation set bằng cách tính posterior probability of default cho individual đó, và classify individual vào default category nếu posterior probability lớn hơn 0.5.
 - iv. Tính validation set error, là fraction các observation trong validation set bị misclassified.

- (c) Lặp process trong (b) ba lần, dùng ba cách chia khác nhau các observation thành training set và validation set. Nhận xét kết quả thu được.
 - (d) Bây giờ xét logistic regression model dự đoán probability of default dùng `income`, `balance`, và một dummy variable cho `student`. Estimate test error cho model này bằng validation set approach. Nhận xét liệu việc thêm dummy variable cho `student` có làm giảm test error rate hay không.
6. Ta tiếp tục xét logistic regression model dự đoán probability of default bằng `income` và `balance` trên dataset `Default`. Cụ thể, bây giờ ta tính estimate standard error của logistic regression coefficient ứng với `income` và `balance` theo hai cách: (1) dùng bootstrap, và (2) dùng standard formula để tính standard error trong function `sm.GLM()`. Đừng quên đặt random seed trước khi bắt đầu analysis.
- (a) Dùng function `summarize()` và `sm.GLM()`, xác định estimated standard error của coefficient ứng với `income` và `balance` trong multiple logistic regression model dùng cả hai predictor.
 - (b) Viết function `boot_fn()` nhận input là dataset `Default` cùng index của các observation, và output coefficient estimate cho `income` và `balance` trong multiple logistic regression model.
 - (c) Theo bootstrap example trong lab, dùng function `boot_fn()` để estimate standard error của logistic regression coefficient cho `income` và `balance`.
 - (d) Nhận xét các estimated standard error thu được bằng function `sm.GLM()` và bằng bootstrap.
7. Trong Sections 5.1.2 và 5.1.3, ta thấy function `cross_validate()` có thể dùng để tính LOOCV test error estimate. Một cách khác là chỉ dùng `sm.GLM()` và method `predict()` của fitted model trong một for loop. Bây giờ bạn sẽ dùng approach này để tính LOOCV error cho một simple logistic regression model trên dataset `Weekly`. Nhắc lại rằng trong classification problem, LOOCV error được cho bởi (5.4).
- (a) Fit logistic regression model dự đoán `Direction` bằng `Lag1` và `Lag2`.
 - (b) Fit logistic regression model dự đoán `Direction` bằng `Lag1` và `Lag2`, dùng tất cả observation trừ observation đầu tiên.
 - (c) Dùng model ở (b) để dự đoán direction của observation đầu tiên. Bạn có thể làm điều này bằng cách dự đoán observation đầu tiên sẽ `Up` nếu $P(\text{Direction} = \text{"Up"} \mid \text{Lag1}, \text{Lag2}) > 0.5$. Observation này có được classify đúng không?
 - (d) Viết for loop từ $i = 1$ đến $i = n$, với n là số observation trong dataset, thực hiện từng bước sau:
 - i. Fit logistic regression model bằng tất cả observation trừ observation thứ i để dự đoán `Direction` bằng `Lag1` và `Lag2`.
 - ii. Tính posterior probability thị trường tăng cho observation thứ i .

- iii. Dùng posterior probability của observation thứ i để dự đoán thị trường có tăng hay không.
 - iv. Xác định prediction direction của observation thứ i có sai hay không. Nếu sai, ghi 1; nếu không, ghi 0.
 - (e) Lấy average của n số thu được ở (d)iv để có LOOCV estimate của test error. Nhận xét kết quả.
8. Bây giờ ta thực hiện cross-validation trên một simulated dataset.
- (a) Generate simulated dataset như sau:
- ```
rng = np.random.default_rng(1)
x = rng.normal(size=100)
y = x - 2 * x**2 + rng.normal(size=100)
```
- Trong dataset này,  $n$  và  $p$  bằng bao nhiêu? Viết model dùng để generate data dưới dạng equation.
- (b) Tạo scatterplot của  $X$  so với  $Y$ . Nhận xét điều bạn thấy.
  - (c) Đặt random seed, rồi tính LOOCV error khi fit bốn model sau bằng least squares:
    - i.  $Y = \beta_0 + \beta_1 X + \epsilon$
    - ii.  $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$
    - iii.  $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon$
    - iv.  $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \beta_4 X^4 + \epsilon$ .

Lưu ý bạn có thể thấy hữu ích khi dùng function `data.frame()` để tạo một dataset duy nhất chứa cả  $X$  và  $Y$ .
  - (d) Lập (c) với một random seed khác và báo cáo kết quả. Kết quả có giống ở (c) không? Tại sao?
  - (e) Model nào trong (c) có LOOCV error nhỏ nhất? Đây có phải điều bạn kỳ vọng không? Hãy giải thích.
  - (f) Nhận xét statistical significance của coefficient estimate khi fit từng model trong (c) bằng least squares. Kết quả này có phù hợp với conclusion dựa trên cross-validation không?
9. Bây giờ ta xét Boston housing dataset từ ISLP library.
- (a) Dựa trên dataset này, hãy đưa ra estimate cho population mean của `medv`. Gọi estimate này là  $\hat{\mu}$ .
  - (b) Đưa ra estimate standard error của  $\hat{\mu}$ . Hãy diễn giải kết quả.  
*Hint:* Ta có thể tính standard error của sample mean bằng sample standard deviation chia cho square root của số observation.
  - (c) Bây giờ estimate standard error của  $\hat{\mu}$  bằng bootstrap. Kết quả so với câu trả lời ở (b) thế nào?
  - (d) Dựa trên bootstrap estimate ở (c), đưa ra 95% confidence interval cho mean của `medv`. So sánh với kết quả thu được bằng `Boston['medv'].std()` và two standard error rule (3.9).  
*Hint:* Có thể xấp xỉ 95% confidence interval bằng formula  $[\hat{\mu} - 2SE(\hat{\mu}), \hat{\mu} + 2SE(\hat{\mu})]$ .

- (e) Dựa trên dataset này, đưa ra estimate  $\hat{\mu}_{med}$  cho median value của `medv` trong population.
- (f) Bây giờ ta muốn estimate standard error của  $\hat{\mu}_{med}$ . Không may, không có formula đơn giản để tính standard error của median. Thay vào đó, estimate standard error của median bằng bootstrap. Nhận xét kết quả.
- (g) Dựa trên dataset này, đưa ra estimate cho tenth percentile của `medv` trong Boston census tract. Gọi quantity này là  $\hat{\mu}_{0.1}$ . (Bạn có thể dùng function `np.percentile()`.) *np.percentile()*
- (h) Dùng bootstrap để estimate standard error của  $\hat{\mu}_{0.1}$ . Nhận xét kết quả.